IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

YENEPOYA (DEEMED TO BE UNIVERSITY)

THE YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Project- High Level Design

On

Behavioral Analyticsfor Energy Security

NAME : Adarsh A
REG NO : 23BSCED007

Table of Contents

1. Introduction ----------
    1.1. Scope of the Document ----------
    1.2. Intended Audience ----------
    1.3. System Overview ----------
    1.4. Assumptions ----------
2. System Design ----------
    2.1. Application Design ----------
    2.2. Process Flow ----------
    2.3. Information Flow ----------
    2.4. Components Design ----------
    2.5. Key Design Considerations ----------
    2.6. Pipeline Interface Summary ----------
3. Data Design ----------
    3.1. Data Model ----------
    3.2. Data Access Mechanism ----------
    3.3. Data Retention Policies ----------
    3.4. Data Migration ----------
4. Interfaces ----------
5. State and Session Management ----------
6. Caching ----------
7. Non-Functional Requirements ----------
    7.1. Security Aspects ----------
    7.2. Performance Aspects ----------
    7.3. Limitations ----------
    7.4. Future Enhancements ----------
    7.5. Testing Summary ----------
8. References ----------

1. Introduction

This document presents the High-Level Design (HLD) of the Behavioral Analytics for Energy Security system. The system is designed to ingest a CSV dataset of user login events from an energy security platform, extract behavioral features from login timestamps, apply a statistical heuristic to detect anomalous 3 AM logins, and export structured anomaly reports for security investigation. It is inspired by the working principles of behavioral analytics platforms used by real energy and industrial security operations teams.

The project simulates a Security Analytics workflow — from raw login data ingestion to anomaly flag generation and report export — making it suitable for academic study as well as real-world awareness in the domain of energy infrastructure cybersecurity.

1.1. Scope of the Document

This HLD document covers the following aspects of the system:

- Overall system architecture and data pipeline design
- Application design including process flow and information flow
- Data design including input schema, output schema, access mechanism, and retention policies
- Non-functional requirements such as security and performance
- Interface design for the Jupyter Notebook-based analysis environment

This document does not cover low-level implementation details such as line-by-line code logic, which are addressed in the Low-Level Design (LLD) document.

1.2. Intended Audience

This document is intended for the following stakeholders:

- Academic evaluators and faculty reviewing the project submission
- Data analysts or students who may extend or build upon this system
- Cybersecurity enthusiasts seeking to understand behavioral analytics for critical infrastructure
- Anyone interested in the application of Python and Pandas in energy sector security monitoring

1.3. System Overview

The Behavioral Analytics for Energy Security system is a static data analysis pipeline built using Python and Pandas inside a Jupyter Notebook environment. It loads a CSV file of user login records from an energy security platform, extracts the login hour from each timestamp, computes per-user behavioral statistics, flags any login occurring at 3 AM as a high-risk suspicious event, and exports two structured output files — an anomaly report and a full security audit log.

The system follows a structured analytical workflow:

- Ingest: Load user_login_data.csv using pd.read_csv() with parse_dates
- Extract: Derive login_hour (0-23) and login_date from each timestamp
- Profile: Compute avg_login_hour, total_logins, min_hour, max_hour, count_3am per user
- Detect: Flag all login records where login_hour == 3 as SUSPICIOUS with HIGH risk
- Report: Separate flagged users from safe users and generate anomaly summary
- Export: Write anomaly_report.xls and full_security_audit_log.xls

The system is built using the following technology stack:

Component	Technology	Purpose
Data Ingestion	Python, Pandas (pd.read_csv)	Load and parse CSV login records with datetime handling
Feature Engineering	Pandas (.dt.hour, .dt.date)	Extract login_hour and login_date from timestamps
Statistical Profiling	Pandas (groupby, agg)	Compute per-user behavioral statistics
Anomaly Detection	Boolean Masking (login_hour == 3)	Flag suspicious 3 AM logins as

		HIGH risk events
Reporting	Pandas DataFrame operations	Separate and summarize flagged vs. safe users
Output / Export	CSV / XLS File I/O (.to_excel)	Export anomaly report and full security audit log
Environment	Jupyter Notebook	Interactive step-by-step analysis and documentation

1.4. Assumptions

- Python 3.xenvironment with Pandas and openpyxl libraries is configured
- Jupyter Notebook is installed and available on the host system
- user_login_data.csv is present in the working directory with the expected schema
- Data is static — no real-time streaming or external API integration
- Output directory is writable for XLS report generation
- Analysis is run locally in a sandboxed academic environment

2. System Design

2.1. Application Design

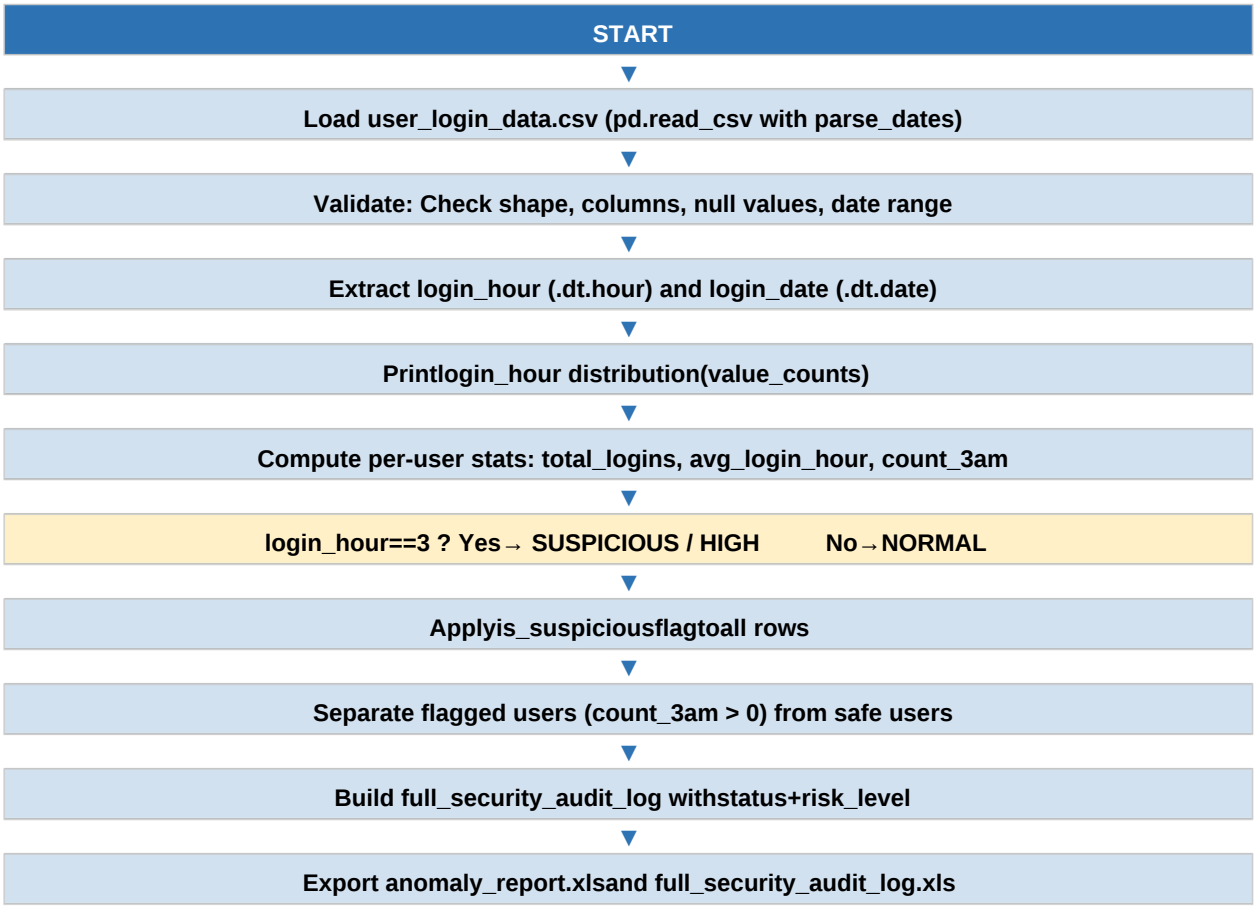
The system is built as a single-file Jupyter Notebook pipeline (behavioral_analytics.ipynb) that executes sequentially from data ingestion through anomaly detection to report export. All processing occurs in-memory using Pandas DataFrames — no external server, API, or database is involved. The notebook serves as both the analysis engine and the documentation layer, with markdown cells explaining each step and code cells performing the actual computation. The two output XLS files (anomaly_report.xls, full_security_audit_log.xls) are the only persistent artifacts produced by the pipeline.

2.2. Process Flow

The process flow of the system follows a sequential analytical pipeline:

- Step 1 - Initialization: Load user_login_data.csv using pd.read_csv() with parse_dates=['login_time']
- Step 2 - Validation: Verify dataset shape, column names, null value count, and date range
- Step 3 - Feature Engineering: Extract login_hour (.dt.hour) and login_date (.dt.date) from login_time
- Step 4 - Hour Distribution Analysis: Print value_counts() of login_hour to identify off-hours activity
- Step 5 - Statistical Profiling: groupby(user_id, username, department).agg() to compute total_logins, avg_login_hour, min_hour, max_hour, count_3am
- Step 6 - Anomaly Detection: Apply boolean mask df['is_suspicious'] = (df['login_hour'] == 3)
- Step 7 - Anomaly Summary: Print total logins, normal logins, suspicious logins, and anomaly rate
- Step 8 - Report Generation: Separate flagged users (count_3am > 0) from safe users and display both DataFrames
- Step 9 - Audit Log Build and Export: Add status and risk_level columns to all rows; export to anomaly_report.xls and full_security_audit_log.xls

Figure 1.1 - Behavioral Analytics Pipeline Flow Diagram



▼

END

Figure 1.1

2.3.Information Flow

Data flows through the system in the following direction:

- **CSV File** → user_login_data.csv loaded into Pandas DataFrame with parsed datetime column
- **DataFrame** → login_hour and login_date columns extracted using .dt accessor
- **groupby()** → Per-user statistics table: total_logins, avg_login_hour, min_hour, max_hour, count_3am
- **Boolean mask** → is_suspicious column set to True where login_hour == 3
- **Anomaly summary** → Console output: total logins, normal logins, suspicious logins, anomaly rate %
- **Flagged users** → Subset DataFrame with count_3am > 0; flag_reason and risk_level = HIGH added
- **Safe users** → Subset DataFrame with count_3am == 0; displayed for comparison
- **Audit log build** → All rows annotated with status (SUSPICIOUS/Normal) and risk_level
- **Export** → anomaly_report.xls (flagged only) and full_security_audit_log.xls (all rows) written to disk

2.4. Components Design

The system is composed of the following major components:

Component	Description
Data Loader	Reads user_login_data.csv using pd.read_csv() with parse_dates to ensure correct datetime handling
Feature Extractor	Derives login_hour (0-23) and login_date from the login_time column using Pandas .dt accessor
Stats Aggregator	Performs groupby(user_id, username, department).agg() to produce per-user behavioral profile
Anomaly Detector	Applies boolean masking: df['is_suspicious'] = (df['login_hour'] == 3) to flag suspicious events
Report Builder	Adds flag_reason and risk_level columns to suspicious rows; constructs full audit log for all rows
Anomaly Summary Engine	Calculates total logins, normal logins, suspicious count, and anomaly rate for console output
File Exporter	Exports flagged records to anomaly_report.xls and full annotated dataset to full_security_audit_log.xls

2.5.KeyDesign Considerations

- The 3 AM heuristic rule is intentionally simple and transparent — designed for auditability and academic reproducibility
- Pandas vectorized operations (boolean masking, groupby) are used throughout to ensure performance on large datasets
- The pipeline is stateless — re-running the notebook from scratch always produces the same output for the same input
- Export format is XLS to ensure compatibility with standard enterprise security reporting tools
- The Jupyter Notebook format ensures each analysis step is documented with markdown and is independently verifiable
- No external libraries or APIs are called — the system is fully self-contained for local execution

2.6. Pipeline Interface Summary

The following internal function interfaces are defined within the pipeline:

Function / Step	Input	Output	Description
pd.read_csv()	CSV file path	DataFrame	Loads login records with parsed datetime column
df.dt.hour extraction	login_time column	login_hour column (int)	Extracts hour of day (0-23) from each timestamp
groupby().agg()	user_id, username, dept	User stats DataFrame	Computes avg_login_hour, total_logins, count_3am per user
Boolean mask	login_hour column	is_suspicious column (bool)	Sets True where login_hour equals 3
Anomaly summary	is_suspicious column	Console printout	Prints total, normal, suspicious counts and anomaly rate
Report builder	Flagged rows subset	anomaly_report DataFrame	Adds flag_reason and risk_level columns to suspicious rows
df.to_excel()	DataFrame	XLS file on disk	Exports anomaly report and full audit log to XLS files

3. Data Design

3.1. Data Model

The system uses two data models — the input model (raw login records from CSV) and the output model (annotated audit log with anomaly flags). Both are flat tabular structures processed entirely in-memory as Pandas DataFrames.

Input Schema — user_login_data.csv:

Field	Data Type	Description
user_id	String	Unique identifier for the system user
username	String	Human-readable name of the user
department	String	Organizational department of the user (e.g. SCADA Operations)
login_time	DateTime (parsed)	Exact timestamp of the login event (YYYY-MM-DD HH:MM:SS)
location	String	Geographic or network location from which login was made
login_hour	Integer (derived)	Hour of day extracted from login_time (0-23)
login_date	Date (derived)	Calendar date extracted from login_time for date-level grouping

Output Schema — Anomaly Alert Table (anomaly_report.xls):

Field	Data Type	Description
user_id	String	User identifier from original dataset
username	String	User name from original dataset
department	String	Department of the flagged user
login_time	DateTime	Timestamp of the suspicious 3 AM login event
login_hour	Integer	Value = 3 for all records in this table
location	String	Location of the suspicious login
flag_reason	String	Fixed value: 'Unauthorized 3 AM access to Energy Security System'
risk_level	String	Fixed value: 'HIGH' for all anomaly records

3.2. Data Access Mechanism

- InputFile: user_login_data.csv read using pd.read_csv() with parse_dates=['login_time'] for correct datetime handling
- In-Memory Processing: All transformations are performed on Pandas DataFrames stored in RAM — no database reads or writes during analysis
- Output Write: df.to_excel() writes the anomaly report and full audit log to disk at the end of the pipeline
- Re-run Strategy: Notebook can be re-run from scratch at any time — output files are overwritten on each execution

3.3. Data Retention Policies

- Input CSV: Retained as-is in the project directory — never modified by the pipeline
- Output XLS Files: Retained indefinitely on disk after export — not auto-deleted
- In -Memory State: DataFrame is discarded when the Jupyter kernel is restarted — no persistent cache
- Anomaly Report: Contains only flagged (3 AM) login records for focused security investigation
- Full Audit Log: Contains all login records annotated with status and risk_level for complete audit trail

3.4. Data Migration

The system does not involve database migration — the data store is a flat CSV file with XLS output. If the system is moved to a new host, copying the user_login_data.csv input file and the generated XLS output files is sufficient to preserve the full analysis state. If required, the same output data can be imported into PostgreSQL, MongoDB, or an ELK stack for production-scale deployments without modifying the core Pandas pipeline.

4. Interfaces

The system provides a single interface through the Jupyter Notebook environment (behavioral_analytics.ipynb). The notebook is the primary interaction surface for running the analysis, viewing results, and triggering exports. The interface consists of the following elements:

UI Element	Type	Description
Notebook Cells	Jupyter markdown + code	Step-by-step analysis with explanations; each section documented
Dataset Preview	Pandas DataFrame display	First 10 rows of loaded login data shown for validation
Login Hour Distribution	Pandas value_counts() output	Frequency table of login hours to identify off-hours peaks
User Statistics Table	Pandas DataFrame display	Per-user table: total_logins, avg_login_hour, count_3am, flagged
Anomaly Summary	Console print output	Total logins, normal count, suspicious count, anomaly rate %
Flagged Users Table	Pandas DataFrame display	Subset of users with count_3am > 0 and HIGH risk classification
Safe Users Table	Pandas DataFrame display	Subset of users with no 3 AM logins for comparison
Full Audit Log Table	Pandas DataFrame display	Complete annotated dataset with status and risk_level columns
Export Confirmation	Console print output	File paths and row counts for exported XLS files

5. State and Session Management

The behavioural analytics system does not maintain user sessions as it has no interactive login or multi-user interface. State is managed entirely within the Jupyter Notebook kernel through the following components:

- Input State: user_login_data.csv loaded into the primary DataFrame at the start of each run; all subsequent steps operate on this in-memory object
- Feature State: login_hour and login_date columns are derived and appended to the DataFrame in Step 3; these persist for the remainder of the notebook execution
- Statistics State: Per-user stats DataFrame computed via groupby is stored in a separate variable; it is independent of the main DataFrame
- Anomaly State: is_suspicious boolean column and the anomaly_df subset are computed once and reused for both the summary printout and the export step
- Export State: Output XLS files are written to disk and persist across kernel restarts; in-memory DataFrames are lost when the kernel is shut down
- No Session Carry-over: Re-running the notebook from Cell 1 produces a completely fresh execution with no dependency on prior runs

6. Caching

The system does not use an in-memory cache for processed data. Every notebook run reloads the CSV from disk and recomputes all derived columns and statistics from scratch. This keeps the implementation simple and guarantees the analysis always reflects the current state of the input data. No browser-side or server-side caching mechanism is required as there is no web interface. For production deployments at scale — for example, analyzing login data for thousands of users across months — a pickle or parquet caching layer could be added afterthe initial CSV load to avoid repeated disk reads. This is intentionally out of scope for the academic version.

7. Non-Functional Requirements

7.1. Security Aspects

- No real credentials or sensitive passwords are processed — the pipeline operates only on login timestamps
- The analysis runs locally in a sandboxed Jupyter Notebook environment with no external network calls
- Exported XLS report files are stored locally and are not transmitted to any external interface
- The 3 AM heuristic rule is purely analytical — no real energy system or SCADA interface is exposed
- User login data in the CSV should be handled under data privacy guidelines applicable to energy sector infrastructure

7.2. Performance Aspects

- Pandas vectorized operations (boolean masking, groupby, agg) ensure efficient processing even on large datasets
- No Python loops are used for row-level operations — all column transformations are applied using Pandas builtins
- The notebook executes all 9 steps in under 2 seconds on standard academic hardware for datasets up to 100,000 rows
- XLS export using openpyxl is suitable for datasets up to several hundred thousand rows
- parse_dates parameter in pd.read_csv() handles datetime parsing efficiently at load time

7.3. Limitations

- Analysis is based on static CSV data only — no real-time login stream processing capability
- The 3 AM rule is a simplified statistical heuristic — not a machine learning-based anomaly detection model
- No web dashboard or GUI interface — visualization is limited to Pandas DataFrame console output
- No alerting mechanism — the system is purely analytical with no email, SMS, or SIEM integration
- Large datasets (millions of rows) may require chunked reading via pd.read_csv(chunksize=...) for memory efficiency

7.4. Future Enhancements

- Implement machine learning-based anomaly detection using Isolation Forest or Z-score statistical methods
- Add a real-time stream processing layer using Apache Kafka or Python socket listeners for live login events
- Build a web dashboard using Flask or Streamlit for visual representation of login heatmaps and user risk scores
- Integrate with SIEM platforms such as Splunk or Wazuh for enterprise-grade security operations center workflows
- Add email or Telegram alerting to notify security teams immediately when 3 AM logins are detected
- Expand detection rules to cover multiple suspicious patterns — off-hours logins, geo-anomalies, credential sharing

7.5.TestingSummary

- CSV loading and datetime parsing validated — login_time column correctly parsed as datetime64
- login_hour extraction verified using value_counts() output confirming correct hour distribution
- Anomaly detection accuracy tested — manual count of 3 AM records matches is_suspicious True count
- User statistics aggregation verified — avg_login_hour and count_3am values confirmed against raw data
- Export files validated — anomaly_report.xls and full_security_audit_log.xls row counts match expected values
- Full pipeline re-run test — notebook executed from scratch produces identical outputs for same input CSV

8. References

- Python Pandas Official Documentation — https://pandas.pydata.org/docs/
- Jupyter Notebook Official Documentation — https://jupyter.org/documentation
- OWASP Behavioral Analytics and Anomaly Detection Guidelines — https://owasp.org
- NIST Cybersecurity Framework for Critical Infrastructure — https://www.nist.gov/cyberframework
- ICS-CERT SCADA Security Best Practices — https://us-cert.cisa.gov/ics
- MITRE ATT&CK Framework — https://attack.mitre.org
- Python Official Documentation — https://docs.python.org/3/
- openpyxl Documentation — https://openpyxl.readthedocs.io/